

KALYANI PRIYA



SMART WEATHER MONITOR PRO

Project Overview

SMART WEATHER MONITOR PRO is an intelligent weather analytics & forecasting platform designed for real-time, accurate, and multi-city weather monitoring on mobile devices.

Problem: Unreliable or delayed weather information disrupts daily planning, travel, agriculture, and emergency response for millions across India.

Need: A unified, real-time mobile solution providing accurate forecasts, multi-city monitoring, and smart weather alerts at your fingertips.

Objectives: Deliver timely weather forecasts & alerts, enable data analytics insights, ensure seamless multi-city tracking, and provide a premium user experience powered by OpenWeatherMap API.



Technologies Used

FLUTTER&UI

Flutter is a cross-platform mobile UI framework by Google, enabling a single codebase for Android & iOS. Combined with Material 3 design system and Android Studio as the development environment, it delivers a premium, responsive, and visually consistent user interface across all devices.

JAVA OOP & API

The backend is powered by Java OOP Architecture ensuring robust, scalable logic through Encapsulation, Inheritance, and Polymorphism. OpenWeatherMap API serves as the real-time weather data source, providing live forecasts, alerts, and multi-city weather data seamlessly integrated into the platform.

DATA & STORAGE

SQLite database manages structured local storage for weather history, user preferences, and cached forecasts. SharedPreferences handles lightweight key-value data for quick access. Together, they enable reliable offline support and fast data retrieval, ensuring smooth performance even without network connectivity.

System Architecture

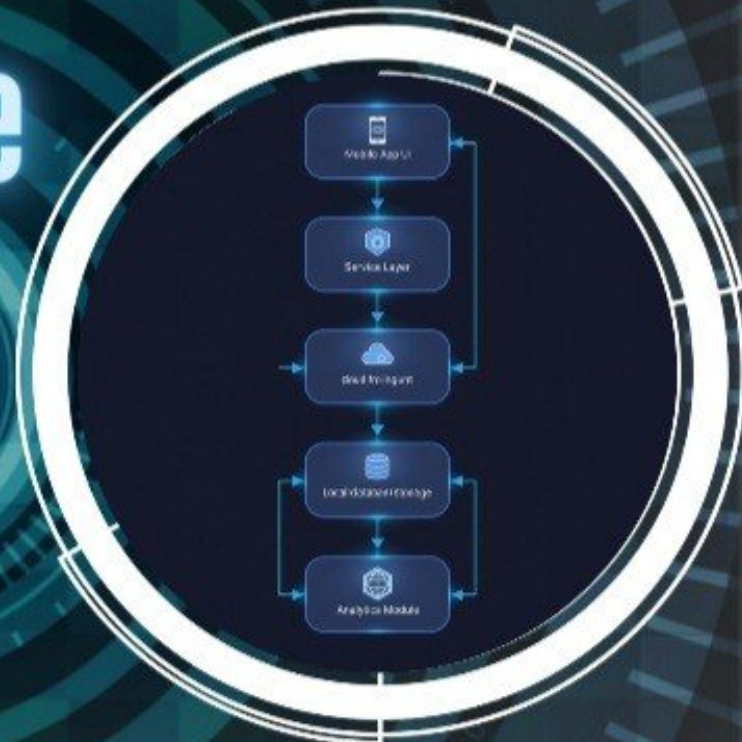
Flutter Mobile App: Cross-platform UI layer capturing user inputs and rendering weather data with Material 3 components.

Weather Service Layer: Intermediary logic module that processes API requests, handles errors, and formats data for the app.

OpenWeatherMap API: Real-time external data source delivering live weather, forecasts, and alert feeds.

Local Storage (SQLite / SharedPreferences): Caches weather data for offline access and stores user preferences securely.

Analytics & Alerts Module: Processes historical and real-time data to generate insights, trend reports, and push notifications for weather alerts.



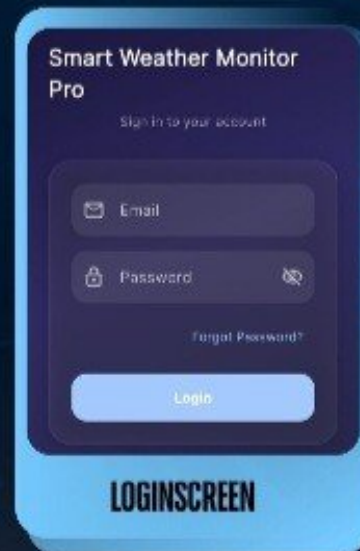
Java OOP Implementation

Key Classes: WeatherData, WeatherService, WeatherManager, AlertManager, AnalyticsManager

- Encapsulation: Securing weather data inside well-defined classes
- Inheritance: Reusing and extending base class functionality
- Polymorphism: Methods behaving differently across subclasses
- Abstraction: Simplifying complex systems via interfaces

- Interfaces: Defining contracts for platform components
- Modular OOP design ensures scalability, maintainability, and clean separation of concerns across all weather service layers.

Application UI Design



Key Features

01

REAL-TIME
MONITORING

02

HOURLY FORECAST

03

WEEKLY FORECAST

04

MULTI-CITY SEARCH

05

WEATHER ALERTS

06

ANALYTICS
DASHBOARD

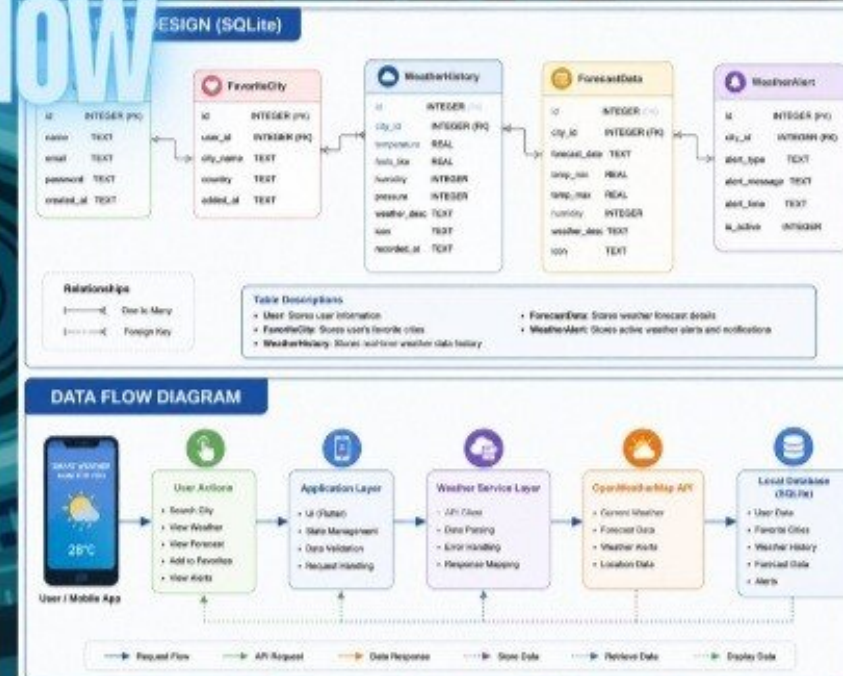
Database & Data Flow

ER Diagram Entities:

- User – stores credentials, preferences & city list
- WeatherData – current conditions per city (temp, humidity, wind)
- ForecastData – hourly & weekly forecast records
- WeatherHistory – archived historical weather logs
- FavoriteCity – user-saved city references

Relationships: User ↔ FavoriteCity (one-to-many) | WeatherData ↔ ForecastData (one-to-many) | User ↔ WeatherHistory (one-to-many)

Data Flow: App UI → WeatherService → OpenWeatherMap API → SQLite/SharedPreferences → Analytics & Alerts Module. Local caching ensures offline availability of last-fetched data.



Results & Testing

API&ACCURACY

Successful integration with OpenWeatherMap API confirmed with real-time data retrieval. High weather data accuracy validated through systematic test cases across multiple cities and weather conditions.

PERFORMANCE&UX

Smooth application performance verified on Android devices with fast load times and stable UI. Positive user experience testing feedback received, confirming intuitive navigation and reliable forecast delivery.

33.9°C

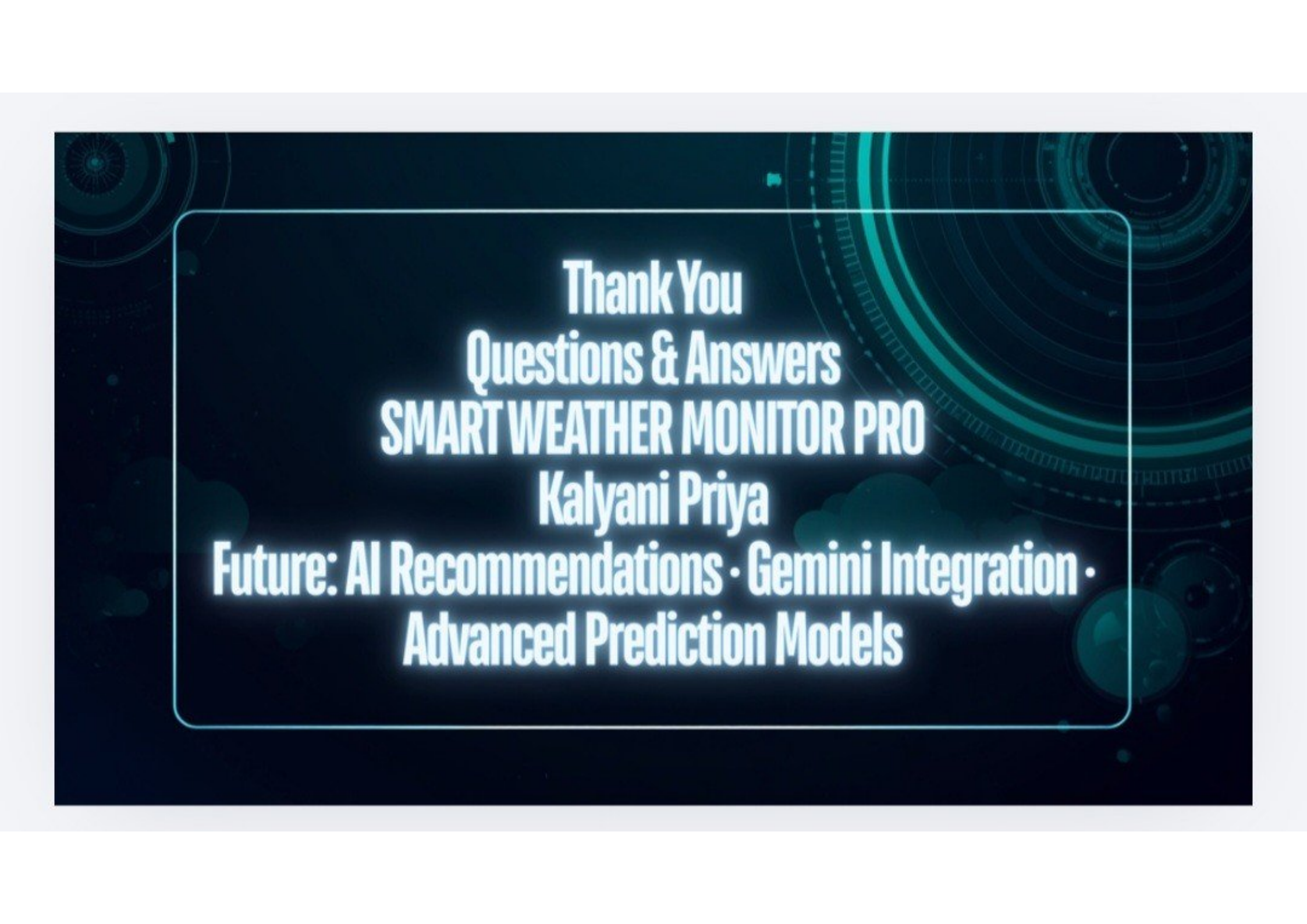
8

Temperature Trend (°C)



Humidity Level (%)

```
1 package java_oop.models;
2
3 import java.util.Date;
4
5 /**
6  * Demonstrates Encapsulation:
7  * - Private fields
8  * - Accessors (Getters/Setters) with validation logic
9  * - Constructor overloading
10 */
11 public class User {
12     private int id;
13     private String name;
14     private String email;
15     private String passwordHash;
16     private String profilePicturePath;
17     private Date createdAt;
18
19     // Default Constructor
20     public User() {
21         this.createdAt = new Date();
22     }
23
24     // Overloaded Constructor (for guest user)
25     public User(String name) {
26         this.name = name;
27         this.email = "guest@local";
28         this.passwordHash = "";
29         this.createdAt = new Date();
30     }
31
32     // Fully Parameterized Constructor
33     public User(int id, String name, String email, String passwordHash, String profilePicturePath, Date createdAt) {
34         this.id = id;
```

Thank You
Questions & Answers
SMART WEATHER MONITOR PRO
Kalyani Priya

Future: AI Recommendations · Gemini Integration ·
Advanced Prediction Models